

HOMework TEMPLATE

Use this template to record your answers for Homework . Add your answers using $\text{\LaTeX}$ and then save your document as a PDF to upload to Gradescope. You are required to use this template to submit your answers. **You should not alter this template in any way** other than to insert your solutions. You must submit all **11** pages of this template to Gradescope. Do not remove the instructions page(s). Altering this template or including your solutions outside of the provided boxes can result in your assignment being graded incorrectly.

You should also export your code as a .py file and upload it to the **separate** Gradescope coding assignment. Remember to mark all teammates on **both** assignment uploads through Gradescope.

Instructions for Specific Problem Types

On this homework, you must fill in blanks for each problem. Please make sure your final answer is fully included in the given space. **Do not change the size of the box provided.** For short answer questions you should **not** include your work in your solution. Only provide an explanation or proof if specifically asked.

Fill in the blank: What is the course number?

10-703

Problem 0: Collaborators

Enter your team members' names and Andrew IDs in the boxes below. If you worked in a team with fewer than three people, leave the extra boxes blank.

Name 1:	Ruiyang Zhou	Andrew ID 1:	ruiyangz
Name 2:	Sangram Kumar Sahoo	Andrew ID 2:	sangrams
Name 3:		Andrew ID 3:	

Add contents here

Introduction

In this assignment we will explore how we can make value-based RL algorithms (in particular TD3 from Homework 3) work in the Offline RL setting - when we have access to a static dataset of environment interactions collected with some behavior policy, and no online access to the environment for learning. In particular, we will implement behavior regularization and Conservative Q-Learning.

Note - Make sure to take a look at the `ReadMe.md` to set up your conda environment for this homework assignment.

Problem 0: Collaborators

Please list your name and Andrew ID, as well as the names and Andrew IDs of your collaborators.

Problem 1: TD3 + Behavior Regularization (50 pts)

In this homework, we will study the Offline RL setting, in which we want to learn a policy given a static dataset of previous interactions with the environment $\mathcal{D} = \{(s, a, r, s')\}$ instead of learning by directly interacting with the environment. Our first step will be understanding what goes wrong if we simply keep using the same algorithms as before.

Problem 1.1: Loading the offline dataset into the replay buffer

The implementations in this homework will build on top of the TD3 code from Homework 3 (you should, however, start from the new starter code for Homework 6). We would like to run the critic and policy updates from TD3 using data uniformly sampled from the offline dataset (as opposed to from the online replay buffer normally used for TD3).

The offline dataset we are going to use comes from [Minari](#), an Offline RL tasks library. In particular, we are going to use [hopper-medium-v0](#), a classic Offline RL task. In `utils.py`, implement `get_buffer_and_environments_for_task`, which loads data from the offline dataset into the replay buffer, and creates the environment in which we will evaluate the policies we train.

Note: for this assignment, we will not collect any online data into the replay buffer (i.e., the only data we will have in the replay buffer will come from the static offline dataset). Therefore, the size of the replay buffer can simply be the number of timesteps in the offline dataset.

Problem 1.2: Running TD3 without any offline learning modifications

Run vanilla TD3 on the `hopper-medium-v0` task with the default hyperparameters, and plot the resulting learning curves including Evaluation Returns and predicted Q-Values for dataset state-action pairs.

In our reference implementation, we get evaluation returns of around 10 and Q-values of more than 3000 by step 50k. These trends of returns and Q-values give us a clue about what is going wrong here. What insights can we draw from evaluation returns being low but predicted Q-values being high? **Questions:**

Command (30 min):

```
python runner.py --agent td3 --total_steps 100000
```

Solution

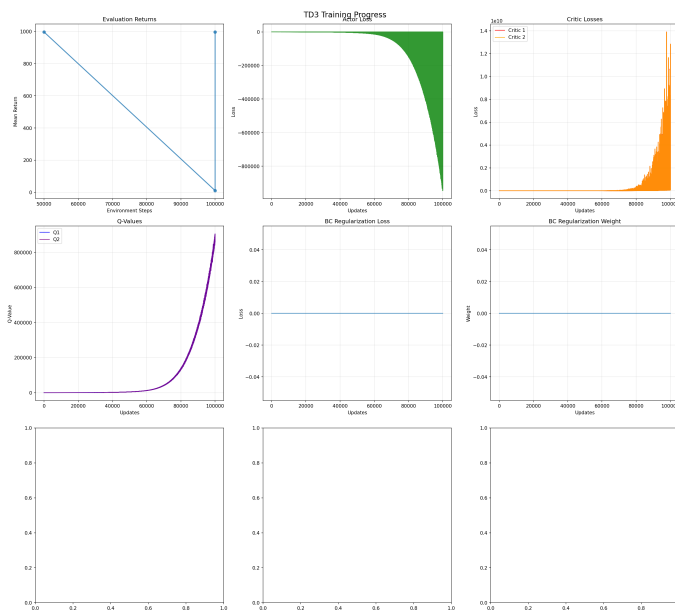


Figure 1: TD3 without any offline learning modifications

High predicted Q-values but very low evaluation returns indicate that TD3 is suffering from overfitting to the training data. This is a sign of a distributional shift.

Problem 1.3: Behavior Regularized TD3

Next, add a behavior-cloning loss to the TD3 policy objective. The policy objective from vanilla TD3 (which comes from DDPG) is given by:

$$\mathcal{L}_{\pi}^{(\text{DDPG})} = -\mathbb{E}_{s \sim \mathcal{D}} [Q_1(s, \mu_{\theta}(s))]. \quad (1)$$

We are going to modify it by adding a Behavior-Cloning term to encourage the policy to

stay close to the actions seen in the offline dataset:

$$\mathcal{L}_{\pi}^{(\text{DDPG} + \text{BC-Reg})} = -\mathbb{E}_{s,a \sim \mathcal{D}} [Q_1(s, \mu_{\theta}(s))] + \alpha_{\text{reg}} \|a - \mu_{\theta}(s)\|_2^2. \quad (2)$$

Implement BC Regularization in `_td3_update_step` inside `td3_agent.py`, and train an agent with $\alpha_{\text{reg}} = 10.000$.

Our reference implementation gets evaluation returns of around 500 at 50k steps.

Solution

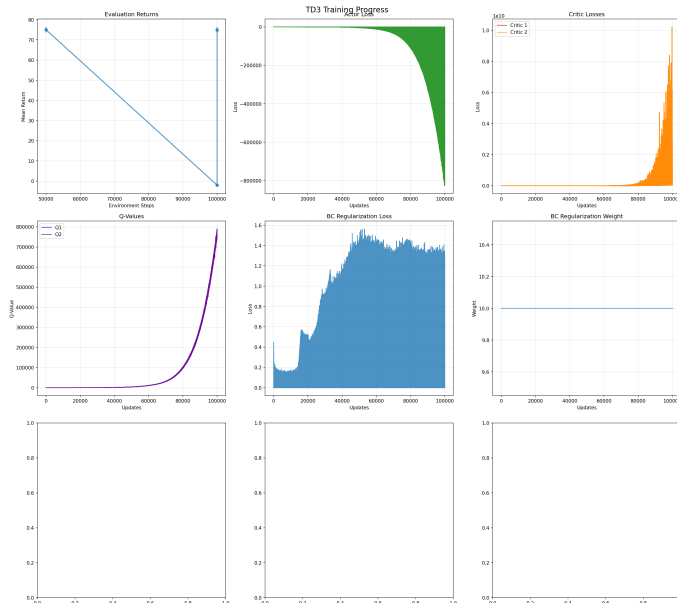


Figure 2: Behavior Regularized TD3

Problem 1.4: Conceptual question about picking α_{reg}

A key choice when implementing DDPG + Behavior Regularization is the choice of the BC-regularization coefficient α_{reg} . Answer the following questions:

1. The effect of the value of α_{reg} is environment and dataset dependent. For which kinds of **datasets** will we expect **larger** values of α_{reg} to work **better**? And conversely, for which datasets will we expect **lower** values of α_{reg} to work better?
2. We would like a systematic way of choosing α_{reg} . Let's say that you first complete a training run with $\alpha_{\text{reg}} = 1.0$, and you have access to any metric you require from this training run; how would you choose a reasonable value of α_{reg} if you could only run a single more run?

Solution

1)

Larger value of α_{reg} work better when the dataset is high-quality, near-optimal and consistent with a single good behavior policy. The reason for that is because strong behavior regularization helps the learned policy stay close to the expert behavior that already performs well.

Conversely, smaller values of α_{reg} work better when the dataset is sub-optimal, diverse, noisy, or contains many off-policy actions. It is because these cases enforcing the policy to mimic bad or inconsistent behavior harms performance.

2)

If the policy stays very close to the dataset actions and the Q-value does not improve, α_{reg} is too large $\rightarrow$ decrease it; if the policy diverges heavily from the dataset behavior or shows unstable Q-estimates, α_{reg} is too small $\rightarrow$ increase it.

Problem 2: Conservative Q-Learning (50 pts)

The behavior-cloning regularization used in Problem 1 constraints the policy to stay close to the actions seen in the dataset, so that the policy can't exploit value overestimations. However, the Q-function itself is left unconstrained, so we might still suffer from this problem to some extent. **Conservative Q-Learning (CQL)** addresses the issue of out-of-distribution over-estimation by **constraining the Q-function** instead of the policy. Please read the CQL paper before completing this problem.

Problem 2.1: Implementing the CQL loss

In `td3_agent.py` `_get_cql_loss`, follow the instructions to implement the CQL regularization loss. In particular, implement the following regularizer that will get added to the standard TD error objective:

$$\alpha \mathbb{E}_{s,a \sim \mathcal{D}} \left[\log \left(\sum_{a' \sim \text{uniform}} (\exp(Q(s, a'))) - Q(s, a) \right) \right] \quad (3)$$

Where a' are actions sampled uniformly inside the action bounds of the environment. Then, run training with CQL alpha 5.

Note: Our reference implementation got around 1.500 returns. However, depending on the implementation some seeds might lead to poor performance. If you don't get at least 1.000, re-run with 2 more seeds.

Solution

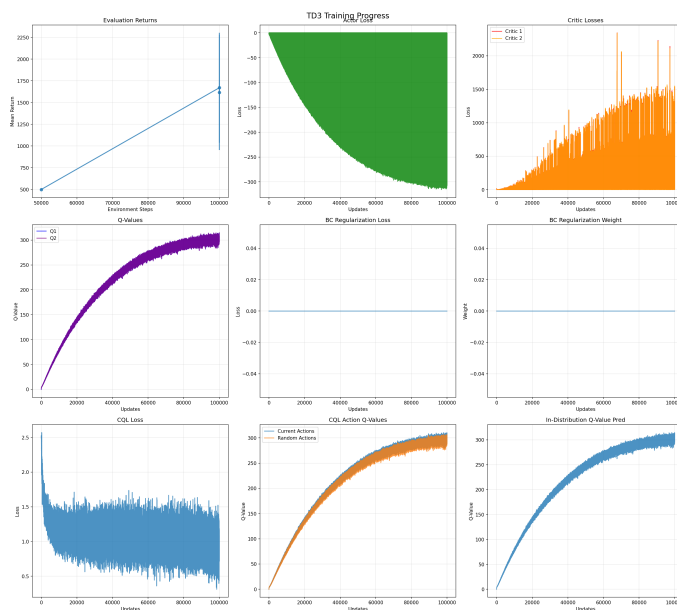


Figure 3: CQL loss

```
(hw6) zeexterxd@Sangrams-MacBook-Pro hw6_starter_code % python runner.py --agent td3 --total_steps 100000 --cql_alpha 5.0

Training completed! Total steps: 100000
Total TD3 updates: 100000
Loaded best model from runs/td3_mujoco/hopper/medium_v0/20251106-122719/best.pt for final evaluation
Final evaluation...
Final performance: 1616.0 ± 663.1
Training curves saved: runs/td3_mujoco/hopper/medium_v0/20251106-122719/training_curves.png
Final training curves saved: runs/td3_mujoco/hopper/medium_v0/20251106-122719/training_curves.png
Final model saved: runs/td3_mujoco/hopper/medium_v0/20251106-122719/final.pt
Training log saved: runs/td3_mujoco/hopper/medium_v0/20251106-122719/training_log.json

All artifacts saved to: runs/td3_mujoco/hopper/medium_v0/20251106-122719
Training complete!
❖ (hw6) zeexterxd@Sangrams-MacBook-Pro hw6_starter_code %
```


Problem 3: Conceptual Questions on Offline (50 pts)

(i) **Select all that apply and justify** regarding the offline RL methods that apply regularization on the value function for training. For this question, consider three algorithms:

- Conservative Q-Learning: as we discussed in class, this algorithm applies a regularizer to minimize large Q-values during training.
- Reward penalty: similar to offline model-based RL methods, we penalize the reward value at *each* state-action pair by subtracting an uncertainty estimate from it.
- Policy constraint penalty: this approach subtracts $D_{\text{KL}}(\pi_{\theta}, \pi_{\beta})$ from the policy update and the value function update.

Select all that apply from below:

1. Conservative Q-learning is likely to perform better than policy constraints on tasks with offline datasets that exhibit higher coverage.
2. Reward penalty should perform better than conservative Q-learning methods because penalizing the Q-function at each unseen action will lead to more conservatism than using uncertainty estimates to penalize the reward function.
3. Conservative Q-learning is addressing problems of over-conservative behavior with policy constraint penalty methods.

Solution

(1) True

CQL tend to have better performance than policy-constraint methods when the offline dataset has high coverage, because policy constraints force the learned policy to stay very close to the behavior policy even when better actions exist in the dataset.

(2) False

Reward-penalty methods do not necessarily outperform CQL; they penalize rewards based on model uncertainty, but this relies heavily on the correctness of the uncertainty estimates. This can make them less conservative or more conservative than intended, depending on how well the model generalizes. CQL, however, is designed specifically to penalize Q-values for unseen or unlikely actions, making it a more reliable mechanism for avoiding overestimation. In this case, the claim that reward penalties should “perform better” is incorrect.

(3) True

CQL addresses the issue of over-conservatism often seen in policy-constraint offline RL methods, which force the policy to remain very close to the dataset’s behavior policy. Instead of restricting the policy directly, CQL regularizes the Q-function to be pessimistic for unsupported actions, allowing the policy to deviate more freely from the behavior policy when the data supports improvement. This helps CQL avoid being overly cautious while still remaining safe in offline settings.

(ii) In class, we studied several methods for offline RL based on policy constraints. Recall that there were two types of methods: one based on distributional constraints (e.g., optimize the Q-function subject to the KL-divergence between the learned policy and the behavior policy) and one based on support constraints (e.g., optimize the Q-function subject to a support constraint between the learned policy and the behavior policy). Describe a scenario where distribution constraint methods would be preferable to support constraints. *Hint:* in

class, we discussed when support constraint methods will be better. Think about when that is not the case and answer this question.

Distribution-constraint methods are preferable when the dataset is diverse, noisy, or contains many exploratory or multi-modal behaviors, because a hard support constraint would be either too loose or too restrictive. In such cases, a soft KL-style distribution constraint lets the policy stay close to the overall behavior distribution while still reweighting better actions and generalizing beyond the exact support.

(iii) **Select all that apply and justify** regarding the performance of model-based offline RL methods that we studied in class.

1. Model-based offline RL methods are generally better than model-free offline RL methods (that do not use a dynamic model), given any offline dataset.
2. Using distributional policy constraints within a model-based offline RL methods can be helpful when the distribution of the offline dataset is **quite narrow**.
3. Using distributional policy constraints within a model-based offline RL methods can be helpful when the distribution of the offline dataset is **quite broad and exhibits high coverage**.

Solution

Model based offline RL methods are not always superior to model free offline RL methods, because their performance strongly depends on the accuracy of the learned dynamics model. When the offline dataset has limited coverage, the model must extrapolate outside the data distribution, leading to large model bias and overly optimistic value estimates. Therefore, model based offline RL is not universally better than model free approaches. However, distributional policy constraints are very helpful when the dataset distribution is narrow, because they restrict the learned policy to remain close to the behavior data, ensuring it only takes actions in regions where the model is accurate. This reduces out of distribution errors and prevents the policy from exploiting inaccuracies in the learned model one of the central challenges in offline RL. In contrast, when the offline dataset is broad and exhibits high coverage, such constraints are generally not needed and may even limit performance. In this case, the model is already reliable across a large portion of the state action space, so restricting the policy to stay close to the behavior distribution unnecessarily reduces the ability of the agent to improve beyond the dataset. Thus, only the second statement is correct.

(iv) Given a fixed static offline dataset of state-action-next state-reward transitions, why can offline RL (not filtered imitation learning) perform better than imitation learning? Give a qualitative example of a dataset where offline RL is expected to outperform imitation and one where imitation is expected to outperform offline RL.

Solution

Offline RL can outperform imitation learning because imitation learning simply tries to copy the behavior in the dataset, without understanding which actions were actually good or bad. In contrast, offline RL uses the reward information within the dataset to evaluate long-term returns, allowing it to “stitch together” the best parts of different trajectories and potentially produce a policy that is better than the original behavior policy. For example, consider a navigation task where the dataset consists mostly of suboptimal, wandering trajectories from a human operator, but occasionally includes short segments where the operator takes an efficient shortcut. Imitation learning would reproduce the dominant wandering behavior because it tries to minimize action prediction error, whereas offline RL would identify that the shortcut yields much higher returns and learn a more efficient policy by emphasizing these high-reward segments. On the other hand, imitation learning will outperform offline RL when the dataset contains clean, near optimal expert demonstrations with very narrow coverage such as demonstrations collected from a highly skilled human or robot where every trajectory is close to optimal. In this case, imitation learning can directly copy the expert’s behavior, while offline RL may struggle due to limited exploration data, sparse rewards, and the difficulty of accurately estimating Q-values without diverse transitions.

Problem 4: Feedback

Feedback: You can help the course staff improve the course by providing feedback. What was the most confusing part of this homework, and what would have made it less confusing?

Time Spent: How many hours did you spend working on this assignment? Your answer will not affect your grade.

Alone	
With teammates	30
With other classmates	
At office hours	